TESTs - Lesson 02 - Building in Remix

What capability of smart contracts is highlighted by the Uniswap example, where it performs currency exchange without a backend or centralized services?

- A Executing complex algorithms off-chain.
- B Performing automated, trustless transactions on-chain.
- C Storing private user data securely.
- D Facilitating traditional banking operations.

When inspecting a transaction on Etherscan, what is worth noting about the "Input Data" section?

- A It contains the complete program code of the smart contract.
- B It represents the parameters for calling a function, not the program itself.
- C It is always human-readable Solidity code.

What is the primary reason high-level languages like Solidity were created for Ethereum?

- A To make it easier for humans to write complex programs that can be compiled into EVM bytecode.
- B To allow direct execution of human-readable code on the EVM.
- C To replace the need for compilers entirely.

When using the Solidity compiler in Remix, what are the two primary outputs it generates from a Solidity source file?

- A Human-readable Solidity code and assembly language.
- B A documentation file and a testing script.
- C An Application Binary Interface (ABI) and EVM bytecode.

What is the purpose of including an SPDX License Identifier at the beginning of a Solidity source file?

A. To specify the required compiler version for the contract.

B. To protect the code from unintended usage and define legal rights.

C. To declare all state variables used within the contract.

Explain the core difference between a `pure` function and a `view` function in Solidity, considering their interaction with the blockchain state.

A. `pure` functions can modify state but cannot read it, while `view` functions can read but not modify. For example, `pure` for calculating local data, `view` for checking contract balance.

B. `pure` functions neither read nor modify the blockchain state. `view` functions can read the blockchain state but cannot modify it.

C. `pure` functions always require gas, while `view` functions are always free to call. Both can modify the blockchain state under specific conditions.

Describe the fundamental process by which human-readable Solidity code is executed on the Ethereum Virtual Machine (EVM).

A. Solidity code is directly interpreted by the EVM.

B. Solidity code is compiled into EVM-compatible bytecode, which the EVM then executes.

C. Solidity code is first converted to assembly, then executed by the EVM.

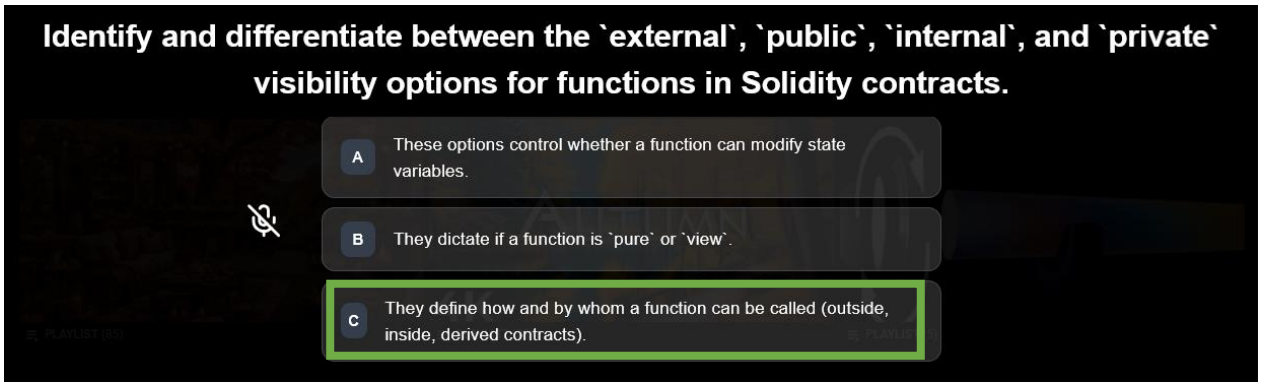
What are the three main state mutability options discussed for Solidity functions, and how do they differ regarding interaction with the blockchain's state?

A. `public`, `private`, `internal`: Controls who can call the function.

B. `pure`, `view`, `payable`: Defines if it reads/modifies state, or accepts Ether.

C. `constant`, `mutable`, `immutable`: Defines if a variable's value can change.

D. `external`, `internal`, `virtual`: Relates to function overriding.


Identify and differentiate between the `external`, `public`, `internal`, and `private` visibility options for functions in Solidity contracts.
A
These options control whether a function can modify state variables.
B
They dictate if a function is `pure` or `view`.
C
They define how and by whom a function can be called (outside, inside, derived contracts).